Selective Pruning for Load-Sensitive Dense Retrieval*

Maria Diana Calugaru[1,*], Federico Siciliano[2,*], Francesca Pezzuti[3,*], Nicola Tonellotto[3,*] and Fabrizio Silvestri[1,*]

[1]Sapienza University of Rome, Italy
[2]National Institute of Geophysics and Volcanology, Italy
[3]University of Pisa, Italy

Abstract

We introduce SELECTIVE, a load-aware selective embedding-pruning method for dense retrievers. Our method adapts the embedding dimensionality at inference time based on per-query deadlines and the current queue occupancy. Under light load, full-dimensional embeddings maximise effectiveness; as load increases, dimensionality is adaptively reduced to maintain throughput and avoid query drops. SELECTIVE is orthogonal to the specific dimensionality reduction strategy employed and operates independently of the embedding model. We evaluate SELECTIVE using both PCA-reduced embeddings and nested Matryoshka embeddings. Our results show that SELECTIVE achieves a better effectiveness–efficiency trade-off than static dimensionality reduction baselines across varying load conditions.

Keywords

Query processing, Dense retrieval, Selective pruning, Efficiency

1. Introduction

Traditionally, retrieval systems have relied on inverted indexes and lexical scoring models such as TF-IDF and BM25 [2, 3]. More recently, dense retrievers have achieved strong effectiveness by mapping queries and documents to a shared high-dimensional vector space, where relevance is estimated through vector similarity [4, 5, 6, 7]. However, similarity scoring is costly at scale [8]. During traffic spikes, full-dimensional scoring can become a bottleneck, saturating processing queues and causing queries to miss their deadlines in latency-constrained scenarios [9].

Prior work has addressed lexical systems' overload through load-sensitive selective pruning [10, 11] and query-efficiency prediction [12, 13], which reduce costs by pruning posting lists or skipping processing stages [14]. Other approaches schedule queries across replicas [15, 16] or adjust CPU frequency [17, 18]. However, these techniques either operate outside the retrieval process or exploit inverted-index-specific properties.

Dense retrieval efficiency, instead, has been improved through graph-based indices [19], vector compression [20], and dimensionality reduction [21, 22, 23, 24, 25, 26]. We focus on dimensionality reduction, which enables simple vector slicing without additional memory overhead or index maintenance. Existing methods include dimension importance estimators (DIMEs) [27], Principal Component Analysis (PCA) [21, 22, 23], and Matryoshka Representation Learning (MRL) approaches [24]. DIMEs improve effectiveness by selecting query-specific important dimensions, but their heavy query-time computations make them unsuitable for latency-critical settings. Low-rank approximations, instead, rely on static, query-agnostic importance scores to compress embeddings and improve efficiency. MRL [24, 25, 26] trains embedding models so that early dimensions capture core semantics. Yet, all apply a fixed dimensionality across queries and therefore do not adapt retrieval cost to system load. We

IIR 2026: 16th Italian Information Retrieval Workshop, November 12–14, 2026, Rome, Italy
*This work is an extended abstract of "Load-sensitive Selective Pruning in Dense Retrieval", accepted at SIGIR 2026 [1].
*Corresponding author.
calugaru@diag.uniroma.it (M. D. Calugaru); federico.siciliano@ingv.it (F. Siciliano); francesca.pezzuti@phd.unipi.it (F. Pezzuti); nicola.tonellotto@unipi.it (N. Tonellotto); fsilvestri@diag.uniroma1.it (F. Silvestri)
0009-0008-5710-3795 (M. D. Calugaru); 0000-0003-1339-6983 (F. Siciliano); 0009-0005-2364-2043 (F. Pezzuti); 0000-0002-7427-1001 (N. Tonellotto); 0000-0001-7669-9055 (F. Silvestri)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

therefore ask: *"Can embedding dimensionality be dynamically adapted to balance retrieval effectiveness and throughput under varying query loads?"*.

We answer this question with SELECTIVE, a load-sensitive selective pruning framework that selects embedding dimensionality for each query according to its deadline and the processing-queue state. When capacity is available, SELECTIVE uses larger representations to favour effectiveness; under saturation, it reduces dimensionality to shorten processing times and prevent query drops. SELECTIVE exploits the progressive structure of PCA and MRL representations, where leading dimensions retain the most salient information, to selectively truncate embeddings at query time.

Our experiments on the MS MARCO passage collection and dev-small queries [28] show that, across varying workloads, SELECTIVE outperforms static pruning baselines, achieving up to +55% MRR or serving up to +400% more queries.

2. Methodology

We now formalise the problem, present SELECTIVE and the progressive representations it relies on.

Problem Definition. Let us consider a server that receives queries $Q = q_1, \dots, q_n$ at times $t_1 \leq \cdots \leq t_n$ and processes them in FIFO order. Queries wait in a queue until service, and each query q_i is subject to a strict latency constraint L. Any query $q_i \in Q$ not served by its deadline $t_i + L$ is dropped.

In dense retrieval, the processing time $\mathscr{C}(d_i)$ of query q_i depends on the number of embedding dimensions d_i used for similarity scoring. Let s_i denote its start time; then, its completion time is $c_i(d_i) = s_i + \mathscr{C}(d_i)$. Since queries are processed sequentially, $s_i = \max(t_i, c_{i-1})$, where c_{i-1} is the completion time of the preceding query. The response time of query q_i is $T_i(d_i) = c_i(d_i) - t_i$. The query is successfully served if $T_i \leq L$, as captured by the indicator function δ_i:

$$\delta_i = \delta_i(d_i) = \mathbb{1}[T_i \leq L] = \mathbb{1}[\mathscr{C}(d_i) \leq (t_i + L) - s_i].$$

Our goal is to find an assignment of dimensions $d_1, \dots, d_n$ to queries in Q, subject to $d_{min} \leq d_i \leq d_{max}$, so as to maximise the total retrieval effectiveness:

$$\max_{d_1,\dots,d_n} \sum_{i=1}^{n} \delta_i \; E(q_i|d_i),$$

where $E(q_i|d_i)$ denotes the effectiveness contribution of query q_i when processed using d_i dimensions. Dropped queries contribute zero because $\delta_i(d_i) = 0$. Here, $d_{\max}$ is the full embedding dimensionality and $d_{\min}$ is a configurable lower bound.

Load-Sensitive Selective Pruning. We propose the SELECTIVE framework, which selects the embedding dimensionality d_i when query q_i starts being processed, based on the current server state. Our method uses an offline-derived cost estimator $\hat{\mathscr{C}}(d)$ that maps each dimensionality to its expected processing time.

Let $\Delta_i = (t_i + L) - s_i$ be the time remaining until the deadline of q_i; if $\Delta_i < \hat{\mathscr{C}}(d_{min})$, we drop q_i since it cannot meet its deadline even using $d_i = d_{min}$. Otherwise, we select the largest d_i that preserves sufficient processing time for the pending queries. Let K be the number of pending queries, including q_i, and let t_{last} be the arrival time of the last enqueued query. Assuming that all K pending queries are processed at $d_{\min}$, we calculate the *available slack* as:

$$\tilde{\Delta}_i = (t_{last} + L) - s_i - K\hat{\mathscr{C}}(d_{min}).$$

If $\tilde{\Delta}_i \leq 0$, we set $d_i = d_{min}$ to avoid dropping pending queries. Otherwise, if q_i can meet its deadline, we allocate $\tilde{\Delta}_i$ to q_i. The adjusted time budget Δ_i^* available for q_i is:

$$\Delta_i^* = \max\left(\min\left(\Delta_i,\ \hat{\mathscr{C}}(d_{min}) + \tilde{\Delta}_i\right), \hat{\mathscr{C}}(d_{min})\right).$$

Finally, exploiting $\hat{\mathscr{C}}(d)$, SELECTIVE selects the largest dimension fitting the available time budget Δ_i^*:

$$d_i = \max_d \{d \mid \hat{\mathscr{C}}(d) \leq \Delta_i^* \wedge d_{min} \leq d \leq d_{max}\}.$$

Note that SELECTIVE is a heuristic rather than a theoretically optimal policy. To be effective, SELECTIVE requires representations that can be efficiently truncated at query time while largely preserving retrieval effectiveness; progressive representations provide this property.

Progressive Representations. Progressive representation methods order embedding dimensions so that earlier dimensions retain most of the information encoded by the full representation. We consider two such methods. PCA [29] projects embeddings onto principal components ordered by decreasing explained variance, enabling fine-grained truncation at any $k \in [d_{\min}, d_{\max}]$. MRL [24] instead learns nested representations optimised at predefined dimensionalities and thus, for coarse-grained truncation.

Combined with a dimension-major index layout, these representations enable effective and efficient query-time pruning in SELECTIVE. Since their leading dimensions retain most of the information in the full representation, SELECTIVE selects the largest dimensionality d_i fitting each query's time budget while limiting effectiveness loss. In the dimension-major layout, these dimensions form a contiguous memory block across the corpus, avoiding random accesses and loads of unused dimensions.

3. Experimental Setup

We now describe the experimental setup we use in our experiments. Our code is available on GitHub[1].

Dataset. In our experiments we use the MSMARCO passage corpus and the *dev-small* query set [28]. We measure effectiveness using the official MRR@10 metric, assigning zero effectiveness to dropped queries. Finally, we measure efficiency as the percentage of queries served within the deadline.

Models & Baselines. We use TAS-B [7] and Contriever [30] pre-trained biencoders. To enable selective pruning via PCA, we apply approximate PCA to the dense index as done in [22], and we project queries in the same vector space. As no public MRL [24] implementation exists for MS MARCO retrieval, we fine-tune a pre-trained BERT [31] encoder using a MRL multi-scale contrastive loss [25] along its final encoder layer with embedding sizes $\{32, 64, 128, 256, 512, 768\}$. As baselines, we use a set of fixed-dimensional baselines offering various effectiveness–efficiency trade-offs. FULL processes every query using $d_{\max}$ dimensions, MIN uses $d_{\min} = 128$, and FIXED uses a fixed dimensionality of 256 or 512.

Indexing. We store the index as a flattened, memory-mapped matrix in a transposed, dimension-major layout. This makes the first k dimensions of all corpus embeddings a single contiguous block, allowing scoring to access only the required values rather than loading unused vector tails.

Simulations & Profiling. We model the system as a single-threaded CPU-based server, as done in other works [18]. We model query arrivals as a Poisson process [32] with arrival rates in $[1, 20]$ queries per second (QPS); this range allows us to evaluate performance under light and heavy loads on our hardware. We set the strict latency constraint for every query to $L = 5$ s; this value is higher than production ones but allows us to simulate saturation and idle conditions. The cost estimator $\hat{\mathscr{C}}(d)$ has been built by profiling the retrieval phase on our hardware (offline). Specifically, for each $d \in [64, 768]$, we measure the latency of similarity scoring and top-K ranking over 100 trials. To mitigate measurement noise, we stored the 99th percentile latency for each d into $\hat{\mathscr{C}}(d)$. We implemented $\hat{\mathscr{C}}(d)$ as a lookup table indexed by d, so that accessing the estimated costs has $O(1)$ cost.

4. Results

We now investigate whether adapting embedding dimensionality selectively, based on the current load, provides a better effectiveness-throughput trade-off than static pruning.

Fig. 1 compares SELECTIVE with the static pruning baselines in terms of MRR@10 and the percentage of queries served within the deadline across different query arrival rates.

Under light load (below 3 QPS), SELECTIVE behaves similarly to FULL across all evaluated embedding models: since sufficient processing capacity is available, it selects the full dimensionality ($d = 768$) and achieves the highest effectiveness. As the arrival rate increases, however, FULL becomes unable to

[1] https://github.com/mariadianacalugaru/selective_pruning

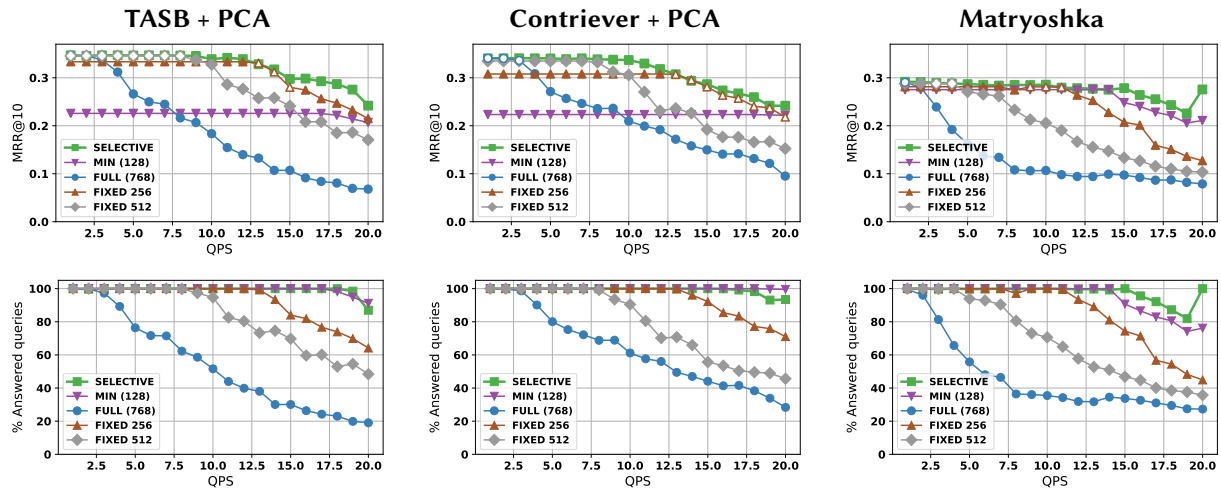


Figure 1: Comparison between selective and static pruning for TASB + PCA, Contriever + PCA and Matryoshka (batch size 16): MRR@10 (top) and percentage of answered queries (bottom) at varying QPS. For MRR@10, filled markers denote significant differences (Wilcoxon signed-rank test with $p < 0.05$) relative to SELECTIVE.

sustain the workload, failing to serve about 50% of queries at 10 QPS and nearly 80% at 20 QPS. At 20 QPS, SELECTIVE serves up to 400% more queries than FULL.

The reduced-dimensional baselines MIN and FIXED sustain higher loads than FULL but sacrifice effectiveness by using the same dimensionality for every query. In particular, MIN serves all queries up to 17 QPS but underutilises the available resources at lower arrival rates. The two FIXED configurations provide intermediate operating points, but no static method provides the best trade-off across all loads.

By adapting dimensionality to the queue state, SELECTIVE uses larger representations when capacity is available and prunes more aggressively as the system approaches saturation. Its effectiveness therefore degrades smoothly as load increases, yielding a better effectiveness-throughput trade-off than the static baselines. This behaviour is consistent across PCA and Matryoshka representations.

5. Conclusions

We introduce SELECTIVE, a load-sensitive pruning method for adapting embedding dimensionality in dense retrieval. Using progressive representations, SELECTIVE adjusts the effectiveness-throughput trade-off at query time to reduce processing costs and meet strict latency constraints. Our results show that SELECTIVE sustains higher query loads than static baselines while preserving retrieval effectiveness. Future work could investigate the theoretically optimal policy under complex workload distributions.

Acknowledgments

This work was partially supported by the FoReLab and CrossLab projects (Departments of Excellence), the NEREO PRIN project (Research Grant no. 2022AEFHAZ) funded by the Italian Ministry of Education and Research (MUR), the NVIDIA Academic Grants Program 2026, and the AI-Sec Lab initiative between Sapienza University of Rome and the CYBER 4.0 Competence Center.

Declaration on Generative AI

During the preparation of this work, the authors used generative AI tools to improve the writing style and check grammar and spelling. The authors subsequently reviewed and edited the content and take full responsibility for the publication.

References

[1] M. D. Calugaru, F. Siciliano, F. Pezzuti, N. Tonellotto, F. Silvestri, Load-sensitive Selective Pruning in Dense Retrieval, in: Proc. SIGIR, 2026.

[2] N. Tonellotto, C. Macdonald, I. Ounis, Efficient query processing for scalable web search, Foundations and Trends in Information Retrieval 12 (2018) 319–492.

[3] C. Manning, P. Raghavan, H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[4] A. Yates, R. Nogueira, J. Lin, Pretrained Transformers for Text Ranking: BERT and Beyond, in: Proc. WSDM, 2021, pp. 1154–1156.

[5] W. X. Zhao, J. Liu, R. Ren, J.-R. Wen, Dense Text Retrieval based on Pretrained Language Models: A Survey, 2022. ArXiv:2211.14876.

[6] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, J.-R. Wen, Large language models for information retrieval: A survey, ACM TOIS (2025).

[7] S. Hofstätter, S.-C. Lin, J.-H. Yang, J. Lin, A. Hanbury, Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling, in: Proc. SIGIR, 2021, pp. 113–122.

[8] B. Cambazoglu, R. Baeza-Yates, Scalability Challenges in Web Search Engines, volume 7, 2011, pp. 27–50.

[9] B. B. Cambazoglu, H. Zaragoza, O. Chapelle, J. Chen, C. Liao, Z. Zheng, J. Degenhardt, Early exit optimizations for additive machine learned ranking systems, in: Proc. WSDM, 2010, p. 411–420.

[10] D. Broccolo, C. Macdonald, S. Orlando, I. Ounis, R. Perego, F. Silvestri, N. Tonellotto, Load-sensitive selective pruning for distributed search, in: Proc. CIKM, 2013, pp. 379–388.

[11] N. Tonellotto, C. Macdonald, I. Ounis, Efficient and effective retrieval using selective pruning, in: Proc. WSDM, 2013, pp. 63–72.

[12] N. Tonellotto, C. Macdonald, I. Ounis, Query efficiency prediction for dynamic pruning, in: Proc. LSDS-IR, 2011, pp. 3–8.

[13] C. Macdonald, N. Tonellotto, I. Ounis, Learning to predict response times for online query scheduling, in: Proc. SIGIR, 2012, pp. 621–630.

[14] D. Broccolo, C. Macdonald, S. Orlando, I. Ounis, R. Perego, F. Silvestri, N. Tonellotto, Query Processing in Highly-Loaded Search Engines, in: String Processing and Information Retrieval, volume 8214, Springer International Publishing, 2013, pp. 49–55. Series Title: Lecture Notes in Computer Science.

[15] A. Freire, C. Macdonald, N. Tonellotto, I. Ounis, F. Cacheda, Scheduling queries across replicas, in: Proc. SIGIR, 2012, pp. 1139–1140.

[16] A. Freire, C. Macdonald, N. Tonellotto, I. Ounis, F. Cacheda, Hybrid Query Scheduling for a Replicated Search Engine, in: P. Serdyukov, P. Braslavski, S. O. Kuznetsov, J. Kamps, S. Rüger, E. Agichtein, I. Segalovich, E. Yilmaz (Eds.), Proc. ECIR, 2013, pp. 435–446.

[17] M. Catena, C. Macdonald, N. Tonellotto, Load-sensitive CPU Power Management for Web Search Engines, in: Proc. SIGIR, 2015, pp. 751–754.

[18] M. Catena, N. Tonellotto, Energy-Efficient Query Processing in Web Search Engines, Trans. KDE 29 (2017) 1412–1425. doi:10.1109/TKDE.2017.2681279.

[19] Y. A. Malkov, D. A. Yashunin, Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs, Trans. PAMI 42 (2020) 824–836.

[20] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, Trans. Big Data 7 (2019) 535–547.

[21] F. Siciliano, F. Pezzuti, N. Tonellotto, F. Silvestri, Static Pruning in Dense Retrieval using Matrix Decomposition, 2024. ArXiv:2412.09983.

[22] X. Ma, M. Li, K. Sun, J. Xin, J. J. Lin, Simple and Effective Unsupervised Redundancy Elimination to Compress Dense Vectors for Passage Retrieval, in: Proc. EMNLP, 2021.

[23] A. Khaledian, A. Ghadiridehkordi, N. Khaledian, PCA-RAG: Principal Component Analysis for Efficient Retrieval-Augmented Generation, 2025. ArXiv:2504.08386.

[24] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder,

K. Chen, S. Kakade, P. Jain, A. Farhadi, Matryoshka Representation Learning, in: Proc. NeurIPS, volume 35, 2022, pp. 30233–30249.
[25] S. Wang, S. Zhuang, B. Koopman, G. Zuccon, 2D Matryoshka Training for Information Retrieval, in: Proc. SIGIR, 2025, pp. 3125–3134.
[26] S. Zhuang, S. Wang, F. Zheng, B. Koopman, G. Zuccon, Starbucks-v2: Improved Training for 2D Matryoshka Embeddings, 2025. ArXiv:2410.13230.
[27] G. Faggioli, N. Ferro, R. Perego, N. Tonellotto, Dimension Importance Estimation for Dense Information Retrieval, in: Proc. SIGIR, 2024, pp. 1318–1328.
[28] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, X. Song, A. Stoica, S. Tiwary, W. Tong, MS MARCO: A Human Generated MAchine Reading COmprehension Dataset, in: Proc. InCoCo@NIPS, 2016.
[29] K. P. F.R.S., LIII. On lines and planes of closest fit to systems of points in space, The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science 2 (1901) 559–572.
[30] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, E. Grave, Unsupervised Dense Information Retrieval with Contrastive Learning, 2022. ArXiv:2112.09118.
[31] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proc. NAACL-HLT, 2019, pp. 4171–4186.
[32] M. Jeon, S. Kim, S.-w. Hwang, Y. He, S. Elnikety, A. L. Cox, S. Rixner, Predictive parallelization: taming tail latencies in web search, in: Proc. SIGIR, 2014, pp. 253–262.